Contents

1 Fundamental: A Field Translation Runtime for Relational DOM Interfaces 2

1.1 Abstract . . . 2
1.2 1. Introduction . . . 3
1.2.1 1.1 The problem: interface state is local . . . 3
1.2.2 1.2 The reframe: a relational field, not a particle background . . . 3
1.2.3 1.3 Contributions . . . 3
1.3 2. Background and related work . . . 4
1.4 3. The reciprocal model . . . 4
1.4.1 3.1 Bodies, agents, and the foundational loop . . . 4
1.4.2 3.2 Three invariants . . . 5
1.4.3 3.3 Field versus force: structure versus cause . . . 5
1.5 4. System architecture . . . 6
1.5.1 4.1 A strictly layered, renderer-agnostic design . . . 6
1.5.2 4.2 The DOM boundary, proven . . . 6
1.5.3 4.3 Scale . . . 6
1.6 5. The frame scheduler and registries . . . 6
1.6.1 5.1 Six ordered phases . . . 6
1.6.2 5.2 Six single-concern registries . . . 7
1.6.3 5.3 Lint: surfacing quiet failures . . . 7
1.6.4 5.4 Phase D: the live runtime . . . 8
1.7 6. The force model . . . 8
1.7.1 6.1 A force is a small contract . . . 8
1.7.2 6.2 Passports: every force declares what it is . . . 8
1.7.3 6.3 Truth modes: honesty as a first-class feature . . . 9
1.7.4 6.4 The catalog, in brief . . . 9
1.7.5 6.5 The Natural Field Translation System . . . 9
1.7.6 6.6 Taxonomy hygiene: lanes that never mix . . . 10
1.8 7. Authoring: one contract, three surfaces . . . 10
1.8.1 7.1 Bodies in markup . . . 10
1.8.2 7.2 Three surfaces, one compiled contract . . . 10
1.8.3 7.3 Intent compilation and patterns . . . 11
1.9 8. The reverse half: emergent interface primitives . . . 11
1.9.1 8.1 Conserved attention . . . 11
1.9.2 8.2 Material typography . . . 11
1.9.3 8.3 Cross-boundary causality . . . 12
1.9.4 8.4 Memory fields . . . 12
1.9.5 8.5 Reduced-motion equivalence (overview) . . . 12
1.9.6 8.6 Case studies (overview) . . . 12
1.9.7 8.7 Signals-first mode: the field as pure computation layer . . . 13
1.10 9. Implementation status and limitations . . . 13
1.10.1 9.1 Shipped versus aspirational . . . 13
1.10.2 9.2 Deliberate limitations (the caveat canon) . . . 13
1.11 10. Discussion . . . 14
1.12 11. Conclusion . . . 14
1.13 Appendix A. Reproducibility and verification . . . 14
1.14 Appendix B. Conversion notes (markdown → preprint) . . . 15

1 Fundamental: A Field Translation Runtime for Relational DOM Interfaces

Status: research draft (preprint, work in progress). Paper 1 of the Fundamental family — the flagship paradigm paper. Claims verified against the codebase and canonical docs as of 2026-06-26. See the series index and *the caveat canon* therein. This is a preprint draft, not canonical product documentation.

Post-verification note (0.8.1, 2026-06-26). Three additions since first draft extend, but do not revise, this paper's argument. `field.addEdge()` adds programmatic typed relationships, broadening the `RelationshipRegistry` story (§5.2) from DOM-derived edges to author-declared ones. `registerOverlay()` lets a consumer register a custom overlay draw pass through the `OverlayRegistry`, and `readParticleChannels()` exposes per-particle channel data for non-visual surfaces — both reinforcing the signals-first / multi-surface posture (§8.7). These ride on a documented wire-format contract for the particle/channel read-out. All are additive API; no claim above is retracted.

Author: Zach Shallbetter **Series:** Fundamental Research Papers, Paper 1 of 8 (the flagship paradigm paper)
Companion papers (domain validators): Reading Field (reading); Evidence Fields (AI trust); Motion Equivalence (accessibility); Host-Driven Runtime (architecture); Portable Field Patterns (authoring); Data as Field Participants (data binding); Explainable Interface Behavior (diagnostics). See the series index.

1.1 Abstract

We present **Fundamental**, a platform-native relational field runtime for the Document Object Model (DOM). Conventional web interface state is *local* and *binary*: an element is hovered or not, open or closed, selected or not, and each component owns its interactive lifecycle in isolation. Fundamental replaces this model with a *shared field context* in which DOM elements, particles, relationships, events, users, layout regions, and data records all participate as typed *agents*. The organizing principle is reciprocal: **elements bend the field; the field bends them back.** An element marked as a *body* exerts force on the matter around it; the field samples the resulting density and writes it back to the element as CSS custom properties and thresholded events, which drive weight, size, color, and position. Because the page and the field share one coordinate space, re-sampled every frame, the invisible structure stays locked to the visible layout through scroll, resize, and reflow.

The contribution of this paper is the *system*: (1) a strictly layered, renderer-agnostic architecture whose core computes field behavior against plain data and is *provably* free of DOM *globals*, enforced by a boundary test with an empty allowlist; (2) a six-phase frame scheduler (`discover → read → compute → state → write → render`) and six single-concern registries that eliminate read-after-write layout thrashing and keep the DOM field loop consistent; (3) a force model that separates *structure* (`field()`) from *cause* (`apply()`) and classifies every behavior by a *truth mode* and an inspectable *passport*; and (4) a uniform authoring contract — one `[data-body]` markup compiling identically across native HTML, web components, and React. We describe the model, the architecture, and the emergent interface primitives the reciprocal loop makes possible (conserved attention, material typography, cross-boundary causality, memory fields), and we are candid about what is shipped versus aspirational. The companion papers each validate the paradigm in one specific domain — reading, evidence and trust, accessibility, runtime architecture, authoring, data binding, and diagnostics.

1.2 1. Introduction

1.2.1 1.1 The problem: interface state is local

The web platform offers three established layers — HTML for structure, CSS for presentation, JavaScript for behavior — and a mature ecosystem of component frameworks on top of them. In all of them, *interactive state is local to a component and largely binary.* A button is hovered or not. A panel is open or closed. A row is selected or not. When a component needs to express relationship, emphasis, history, or causality, it does so through ad hoc layout, color, class names, and one-off logic. The relations are real — this card *depends on* that one; this result is *more relevant* than its neighbors; this section has *already been read* — but they live nowhere in particular and are re-derived, inconsistently, wherever they happen to surface.

Fundamental starts from a different question. Instead of asking *"what animation should this component play?"* it asks *"why is this element emphasized? what is pulling attention? which relationship is active? what caused this state?"* The answer is to treat the interface not as a collection of isolated components with decorative transitions, but as **one shared, inspectable field of meaning** in which those relations are first-class, measurable, and reciprocal.

1.2.2 1.2 The reframe: a relational field, not a particle background

Particle backgrounds are a familiar web decoration: a canvas of dots reacts to the cursor and sits *on top of* the page. Fundamental inverts this in two ways. First, the coupling is **two-way and bound to the layout** — a *DOM field runtime* loop, not a *DOM canvas* one. Real elements exert real force; the field writes real density back into them. Second, the canvas is demoted: it is *one render surface* among several (canvas, SVG overlays, CSS-variable feedback, diagnostic layers), not the system itself. The system is the shared field context that connects them.

This reframe is load-bearing throughout the paper. We will repeatedly distinguish the *field* (the invisible relational structure) from any one *rendering* of it, and the *core* (which computes the field against plain data) from the *platform* (which binds it to the DOM).

1.2.3 1.3 Contributions

This paper contributes the **system** behind Fundamental:

1. **A reciprocal model of interface state** (§3): three invariants — *reciprocity*, *conservation*, *synchronization* — under which elements and field continuously act on each other within one coordinate space.
2. **A renderer-agnostic layered architecture** (§4) whose core is *provably* free of DOM globals (a boundary test runs with an empty allowlist), making the engine portable to any render target while the browser binding lives entirely in a separate platform layer.
3. **A frame scheduler and registry design** (§5): six ordered phases and six single-concern registries that enforce read/write discipline, so reads never thrash against writes and the loop stays consistent across scroll, resize, and reflow.
4. **A force model with explicit epistemics** (§6): the `field()`/`apply()` split (structure vs cause), per-force *passports*, and a six-way *truth-mode* taxonomy that keeps the system honest about which behaviors are physics, which are design, and which are expressive.
5. **A uniform authoring contract** (§7): one `[data-body]` markup that compiles identically across native HTML, web components, and React, plus an intent compiler and a conformance-gated *pattern* format.
6. **Emergent interface primitives** (§8) that the reciprocal loop yields and that have no direct analogue in CSS or component state: conserved attention, material typography, cross-boundary causality, and memory fields.

This paper *names the paradigm* and presents the model, the architecture in overview, the taxonomy, the Natural Field Translation System (§6.5), the pattern runtime, and accessibility. The domain-specific validations follow in their own papers, one claim each: reading (Paper 2), evidence and trust in AI interfaces (Paper

3), reduced-motion equivalence (Paper 4), the host-driven runtime architecture in depth (Paper 5), portable pattern authoring (Paper 6), data binding (Paper 7), and explainable diagnostics (Paper 8). Throughout, we mark shipped behavior distinctly from aspirational or opt-in behavior (§9).

1.3 2. Background and related work

Fundamental sits at the intersection of several lines of work; we position it against each and develop the conceptual lineage of the Natural Field Translation System in §6.5.

Physical metaphors and force-directed layout. Force-directed graph drawing [eades1984; fruchterman1991] and its web embodiment in d3-force [bostock2011; d3force] use simulated attraction and repulsion to *position* nodes. Fundamental shares the simulated-force substrate but differs in intent and in coupling: forces here are not only a layout solver but a continuous *expressive medium* that writes state back into arbitrary semantic elements, and the bodies are the page's real DOM rather than an abstract node set. Self-laying-out layout is one capability among many, not the purpose.

Stigmergy and field-based coordination. Several Fundamental primitives — `diffuse`, `propagate`, `memory` — are scalar-field mechanisms with antecedents in stigmergy and pheromone-field models [grasse1959; dorigo2006]: agents deposit into a decaying grid and later read its gradient. We adopt the mechanism as an interface substrate (e.g. a reading trail that makes worn paths attract attention) rather than as an optimization technique.

Attention and salience. Computational saliency [itti1998] models *where* the eye is drawn. Fundamental treats attention as a *conserved, continuous* field over interface elements — *"selection is a decision; attention is a field"* — with an explicit budget (§8.1). The relevant HCI literature on attention, interruption, and alert fatigue frames the motivating claims for the empirical papers (Papers 2–3). [itti1998; bailey2006; mark2008]

Motion, "material" interfaces, and accessibility. Motion-design systems give interface transitions a sense of physical material. Fundamental pushes the metaphor from *transitions between states* to *a continuous medium the interface lives inside*, and treats reduced motion not as a degraded path but as the same field *differently revealed* — developed in Paper 4. [material-design-motion; prefers-reduced-motion; wcag-motion; vestibular-accessibility]

The rendering substrate. Fundamental is built on fundamental web platform primitives (Custom Elements, Shadow DOM, CSS custom properties, Canvas 2D, `ResizeObserver`, `IntersectionObserver`) and anticipates compositor-native bridges (`CSS.registerProperty`, scroll-driven animation, View Transitions) [csshoudini; scrollanim; webcomponents]. The core ships with *zero runtime dependencies*; framework integrations are adapters, not requirements.

The distinguishing stance, across all of these, is **epistemic**: Fundamental classifies every behavior by truth mode and ships an inspectable passport and conformance scenario for each force, so the system is auditable against its own claims (§6, Paper 5).

1.4 3. The reciprocal model

1.4.1 3.1 Bodies, agents, and the foundational loop

A **body** is a registered origin of influence — *"a registered interface object that can originate influence or participate in the field."* The default body is any DOM element carrying a `data-body` attribute, whose geometry is its bounding rectangle; bodies may also be custom-element hosts, Shadow DOM components, React nodes, canvas-local virtual bodies, data records, layout regions, or relationship nodes. A body must expose identity, a geometry provider, force attributes or registered behavior, a field target, a write-back target, debug metadata, and a conformance path. A maxim governs the boundary between presentation and physics:

Presentation may be private. Physical participation must be public, registered, measurable, and testable.

An **agent** is anything that can receive influence, hold state, change behavior, or affect another participant. Particles are only one agent class; the type union is

```
type FieldAgent =
  | ParticleAgent | ElementAgent | RelationshipAgent
  | EventAgent | UserAgent | LayoutAgent | DataAgent;
```

This generalization is what lets "relational behavior runtime" mean more than a slogan; it is developed in §8 and in the companion papers (Papers 2–3). The foundational loop is:

```
DOM body
  → emits field / applies force
  → moves or influences agents
  → accumulates density, heat, memory, entropy, attention
  → writes metrics back to the DOM
  → changes element style, behavior, or state
  → reshapes the field
```

This loop, the canonical concept document states, *"is the core product."*

1.4.2 3.2 Three invariants

The model is disciplined by three invariants that every force is judged against:

1. **Reciprocity.** Elements move matter; matter moves elements. Neither half is purely decorative. The reverse half is concrete: density returns to elements through `--field-density` (local density), `--load` (a sink's accretion fill), and `--lit` (cross-boundary spillover).
2. **Conservation.** *Nothing is created from nothing.* The default field conserves particle count; particles are captured, released, detached, and reclaimed but never spawned or deleted in steady state. Sources and sinks (`spawn`, `sink`) break conservation *only when explicitly budgeted.* This is the system's *strongest* guarantee; energy and momentum are deliberately weaker (§9.2).
3. **Synchronization.** The page and the field share one coordinate space. Each body's `getBoundingClientRect()` is re-sampled onto the field every frame, so the invisible structure stays locked to the visible boxes through scroll, resize, and reflow. A consequence the introduction already flagged: *animating the DOM animates the simulation for free.*

1.4.3 3.3 Field versus force: structure versus cause

The most consequential distinction in the model separates the *structure* a body radiates from the *effect* it has on matter:

```
field(b, x, y)    = invisible structure        (side-effect free)
apply(b, p, env) = the actual cause / effect   (mutates a particle)
```

`field()` powers field-line rendering, streamline tracing, heatmaps, probes, field-aligned transport (`fieldflow`), debug overlays, and topology analysis. `apply()` produces velocity changes, acceleration, curvature, capture, binding, emission, decay, heat, and state change. The slogan is:

A field line is not always a particle path.

The clearest illustration is electromagnetism. The model insists on three separate behaviors — *Electric fields push. Magnetic fields bend. Fieldflow carries.* — and forbids collapsing them: `magnetism.apply()` must remain a Lorentz force (perpendicular to velocity, doing no work, preserving speed), curving charged matter *across* field lines; transport *along* field geometry is the job of `fieldflow`, not magnetism. This is non-negotiable and conformance-tested (Paper 5). The full field/force law table — gravity as softened inverse-square, charge as Coulomb, magnetism as Lorentz, fieldflow as field-aligned steering — is summarized in §6.5 and given in full in the engine reference.

1.5 4. System architecture

1.5.1 4.1 A strictly layered, renderer-agnostic design

Fundamental is organized into packages whose dependency direction is strict and one-way:

```
Fundamental      renderer-agnostic field/force/particle/metric/diagnostic/conformance engine
@fundamental-engine/dom  DOM participation: measurement, state, feedback, relationships,
                    visual bindings, overlays, scheduling, linting
@fundamental-engine/elements  <field-root> / <field-cell> custom elements + [data-body] authoring
@fundamental-engine/react     <FieldField> component + useFieldField hook
@fundamental-engine/vanilla   the framework-free FieldField class

elements → platform → core      react → platform → core      vanilla → platform → core
```

The layering encodes a claim, not merely a code organization: *field behavior is independent of its presentation substrate.* `Fundamental` computes field behavior against plain data and touches no DOM globals; the browser environment adapter — `browserHost()`, viewport/scroll/`requestAnimationFrame` access, the canvas factory, DOM download helpers — lives in `@fundamental-engine/dom`. The core reaches the environment only through an injected `FieldHost` interface, so the same engine runs against Canvas, WebGL, WebGPU, a native target, or a headless harness by supplying a different host.

1.5.2 4.2 The DOM boundary, proven

The renderer-agnostic claim is not asserted; it is *enforced.* `packages/core/src/engine/dom-boundary.test.ts` scans every source file in the core against a denylist of DOM globals (`document.querySelector`, `window.innerWidth`, `requestAnimationFrame`, `new ResizeObserver`, …) and **runs with an empty allowlist** — meaning no core file is permitted any DOM *global.* (The core still operates on DOM nodes that are *injected* into it — the body scanner walks a host-supplied root — but it reaches no DOM global of its own; every environment touchpoint goes through the injected `FieldHost`.) The test further asserts that field behavior *computes with no document present.* Canvas is therefore treated, structurally, as one render surface rather than a core dependency. We regard this test as the architectural keystone: it converts "renderer-agnostic" from a design aspiration into a continuously checked property.

1.5.3 4.3 Scale

For a sense of proportion: the core is on the order of 80 source modules (~12.6k non-test lines) plus ~55 test modules; the platform is ~16 source modules (~1.6k non-test lines) plus its tests. The system implements 36 forces, 8 cosmological presets, 5 global formations, 16+ render/diagnostic modes, 6 platform registries, and a 6-phase scheduler, with 560+ deterministic tests across the five packages (core, platform, elements, vanilla, plus the scheduler/lint suites within platform). (Exact counts drift with the code; these are orders of magnitude as of the verification date.)

1.6 5. The frame scheduler and registries

1.6.1 5.1 Six ordered phases

The platform runs one shared loop with explicit, ordered phases so that registries never interleave reads and writes and thrash layout:

```
discover → read → compute → state → write → render
```

1. **discover** — register/unregister bodies, relationships, and visual bindings (structure changes).

2. **read** — measure the DOM once: snapshot every registered element's geometry and visibility. *This is the only place layout is read.*
3. **compute** — run field/force/agent logic against the immutable snapshot. No DOM access.
4. **state** — fold results into the state registry and apply thresholds. Internal truth, no DOM writes.
5. **write** — flush state to CSS custom properties, data attributes, `ElementInternals` state, and thresholded events.
6. **render** — draw overlays, field lines, and heatmaps from the registries (read-only).

The ordering is fixed and the discipline is non-negotiable: *no DOM writes during read or compute; no geometry reads during write; every registry declares its phase and runs only in that phase.* The practical payoff is the elimination of read-after-write tearing and forced reflow: the loop reads all geometry once into an immutable snapshot, computes against it, and flushes all writes together. This is why the synchronization invariant (§3.2) is cheap to maintain even as the DOM animates.

`createFieldPlatform(root, { strict? })` constructs the runtime: it wires measurement to the `read` phase and feedback flush to the `write` phase, installs a read-phase guard (a measurement requested off-phase is *recorded* by default, or *thrown* under `strict`), and exposes `.scheduler` plus `.on(phase, handler)` so callers can hang their own discover/compute/state/render work off the same loop. A single `tick(now, viewport)` runs one frame and returns a report of which phases ran and any violations.

1.6.2 5.2 Six single-concern registries

Each registry owns exactly one kind of DOM participation and one phase:

Registry	Concern	Phase
`MeasurementRegistry`	frame-stable geometry/visibility for bodies	read
`StateRegistry`	typed, observable element state (number/bool/string/vector2)	compute / state
`FeedbackRegistry`	CSS-variable and thresholded-event write-back	write
`RelationshipRegistry`	typed graph of connections between bodies	compute
`VisualBindingRegistry`	binds expressive visual layers to semantic sources	write
`OverlayRegistry`	relationship/field-line/diagnostic overlays	render

Three points are worth drawing out. First, `StateRegistry` holds *internal truth* distinct from ARIA or attributes; only the `FeedbackRegistry` is permitted to turn that state into DOM-visible output, which keeps a single writer for CSS variables and avoids races. Second, `RelationshipRegistry` encodes the observation that *the DOM is a tree, but interfaces are graphs*: it normalizes the relationships the platform *already expresses* — `a[href#id]`, `label[for]`, `aria-controls` / `-describedby` / `-labelledby` / `-flowto`, and `data-field-relation` / `-target` — into one typed relationship graph mapped onto core `RelationshipAgent`s. Third, `OverlayRegistry` and `VisualBindingRegistry` enforce that *overlays reveal; they do not define*, and that an expressive visual layer is never the sole carrier of meaning — the accessibility seam developed in Paper 4.

1.6.3 5.3 Lint: surfacing quiet failures

`lintPlatform(platform)` aggregates read-only guardrail rules that surface failures which would otherwise be silent data pathologies rather than type errors:

```
relation-target-missing   a [data-field-relation] points at a missing/unregistered body
state-unregistered        an element holds field state but was never registered
overlay-without-links      a relationship overlay has no relationships behind it
feedback-non-css-var       a binding writes ARIA/attributes instead of --field-*
measurement-off-phase      layout was read outside the read phase
visual-orphan / -not-hidden  a visual binding has no semantic body, or a decorative
                             layer is not aria-hidden - accessibility hazards
```

Crucially, *lint reads; it never mutates state, physics, or the DOM.* It is the platform's self-audit, run in the same spirit as the conformance suite for forces (Paper 5).

1.6.4 5.4 Phase D: the live runtime

Since the project's "Phase D," the platform runtime is the **default** for `<field-root>`. The legacy engine still simulates and renders the canvas surface, but the platform now owns DOM participation: body discovery and measurement route through `MeasurementRegistry`; CSS-variable and event feedback route through `FeedbackRegistry` (the eased density signal is unchanged — only the *write target* moved); Shadow-DOM host registration and the relationship graph are platform-owned. Authors can opt back to pure-legacy behavior with `experimental-platform="off"` or `usePlatformRuntime(false)`. The flagship demonstration is the **Reading Field** (an ordinary article — sections, headings, citations, a table of contents — wired to `createFieldPlatform` and exercising all six registries on the scheduler), which we use as a running example and study substrate in Paper 2.

1.7 6. The force model

1.7.1 6.1 A force is a small contract

In code, a force is a registry entry implementing a compact interface (defined in `packages/core/src/engine/types.ts`, registered via `registry.ts`; simplified here):

```
interface Force {
  token: string;
  label: string;
  apply(b: Body, p: Particle, env: Env): void;            // the cause: mutate a particle
  modify?(b: Body, p: Particle, env: Env):                // modifiers scale/gate siblings
    { strength?: number; gate?: boolean };
  source?(b: Body, env: Env): void;                       // budgeted creation via env.spawn()
  field?(b: Body, x: number, y: number): Vec2;            // the structure, for visualization/fieldflow
  scalarField?(b: Body, x: number, y: number): number;    // potential/density for heatmaps
  kinematic?: boolean;                                    // velocity-replacing vs additive (mass scali
}
```

`apply()` is the common case and is called per particle per body each frame, mutating only velocity, heat, phase, capture state, identity, or life. `field()` is optional and *side-effect free*: it returns the invisible structure used by field lines, streamlines, and `fieldflow`. The split from §3.3 is thus a literal interface boundary, not just a conceptual one.

Bodies expose static authoring fields (force `tokens`, `strength`, `range`, `angle`, `spin`, a `when` condition gate, a `feedback` flag, `shaped` sampling, variable-font bounds `fmin`/`fmax`, source mass `M`) and runtime fields refreshed each frame (`cx, cy, hw, hh` geometry, engaged state `on`, on-screen `vis`, per-frame density tally `count` and its eased value `d`, sink load `accreted`). Force tokens *compose*: `data-body="sink attract"` applies both to the same particle.

1.7.2 6.2 Passports: every force declares what it is

> **Every force needs a passport.**

A *passport* (`packages/core/src/contracts/passport.ts`) is a machine-readable declaration that makes a force's physics legible and auditable. It records structural facts (family — canonical/natural/extended; class; truth mode) and physics semantics: *does it own a `field()`? does it read `env.fieldAt()`? does it move particles? does it do work? does it conserve speed? does it require charge or velocity? does it affect neutral matter?* — alongside visual hints (best render modes, common composites). Passports are cross-checked against the live registry and the conformance experiments, so a passport cannot silently drift from the force it describes. The passport is the unit that makes the *verification* story (Paper 5) possible.

1.7.3 6.3 Truth modes: honesty as a first-class feature

Every behavior is classified by a **truth mode**, shipped as a `TruthMode` union and `TRUTH_MODES` catalog:

Truth mode	Meaning	Examples
Physical	modeled after a recognizable physical law	`gravity`, `charge`, `magnetism`
Designed	shaped for readable, stable UI behavior	`attract`, `repel`, `tether`
Hybrid	a designed primitive operating over natural field geometry	`fieldflow`
Diagnostic	reveals internal state	force vectors, heatmaps, inspectors
Poetic	an expressive composite of stable primitives	`blackhole`, `star`, `nebula`
Semantic	maps data/interface meaning into physics	attention, memory, relation fields

This taxonomy is the system's epistemic backbone: it makes Fundamental *honest about which parts are physics, which are design, and which are expressive composites.* It is also load-bearing for correctness. The model deliberately refuses to unify the designed `attract` (a bounded, finite-range UI well, falloff `(1 - d/r)`) with the physical `gravity` (softened inverse-square, $F = GM \cdot \hat{r}/(d^2 + \epsilon^2)$). Keeping the two registers explicit is what lets a typical page stay cheap and legible while a cosmology or physics surface gets real fidelity, *without either contaminating the other.* §6.5 develops the field-translation taxonomy; the salient architectural point here is that the classification is *data*, validated against code, not prose.

1.7.4 6.4 The catalog, in brief

The 36 forces fall into three families:

- **Canonical (9):** `attract`, `repel`, `swirl`, `stream`, `viscosity`, `jet`, `tether`, `wall`, `sink` — designed interface verbs with bounded, legible falloff.
- **Natural (8):** `gravity`, `charge`, `magnetism`, `thermal`, `collide`, `diffuse`, `propagate`, `memory` — real field laws (softened inverse-square, Coulomb, Lorentz, Langevin agitation, elastic collision, diffusion, the wave equation, and a decaying occupancy grid).
- **Designed-extended (19):** `lens`, `gate`, `buoyancy`, `shear`, `crystallize`, `align`, `wind`, `cohesion`, `pressure`, `link`, `morph`, `hunt`, `spawn`, `pigment`, `fieldflow`, `screen`, `warp`, and the `resonate` / `spotlight` modifiers.

Nine **presets** (`blackhole`, `whitehole`, `wormhole`, `star`, `quasar`, `galaxy`, `nebula`, `tornado`, `fountain`) compose primitives into cosmology *with no new engine code*: a preset expands one DOM element into several co-located virtual bodies, each a primitive with its own attributes (e.g. `blackhole = attract + swirl + sink + lens`). This is a compositional algebra over the catalog, and it keeps the registry small while the expressive surface stays large. The per-force math, the class taxonomy (`[A]` body→particle, `[B]` particle particle, `[C]` field-buffer, `[D]` target-geometry, `[S]` source/sink, modifiers), and the integrator are detailed in the engine reference.

1.7.5 6.5 The Natural Field Translation System

The catalog is not a flat list; it is organized by a conceptual layer that is the system's signature intellectual move. Fundamental *does not copy physics into the interface — it translates* the four fundamental fields of physics into interface grammar:

- **Gravity → priority** (convergence, hierarchy, ranking, anchoring, attention wells)
- **Electromagnetism → polarity and signal** (contrast, opposition, routing, field lines, flow)
- **Strong interaction → binding** (groups, clusters, relationship strength, cohesion, lattices)
- **Weak interaction → transformation** (decay, release, mutation, expiration, handoff)

The argument is that *every interface already has* priority, polarity, binding, and transformation, and that physics already has a compact, well-worn language for exactly those relations; Fundamental borrows the language, not the literal physics. The governing doctrine is a single discipline:

Natural fields are not tokens; tokens are translations.

So `gravity` is *both* a fundamental field and an implemented primitive, but the designed verb `attract` is **not** gravity (`attract` is a bounded UI well); `charge` is an electromagnetic translation rather than a fifth fundamental force; `memory` is a persistence *metric*, not a force; and `fieldflow` is *transport along field structure*, not a field of its own. Each behavior then has one clean place in a hierarchy:

`Natural field → interface translation → engine primitive → metric → diagnostic → pattern`

Crucially, this classification is *data*, not prose: it lives as `FORCE_KIND`, `FORCE_FIELD`, and `NATURAL_FIELDS` in `packages/core/src/config/manual.ts`, so the force manual, the live badges, and the documentation all read from one source and cannot drift. It changes *no* particle or engine behavior — it only organizes how each token is explained. The frame the system presents is therefore not "we have physics-inspired UI effects" but: **Four fields. Many expressions. One DOM runtime.**

1.7.6 6.6 Taxonomy hygiene: lanes that never mix

The translation system only stays honest if the vocabulary is kept in strict, non-overlapping lanes. Fundamental separates:

Lane	What it holds	Examples
Concepts	the four conceptual natural fields	gravity, electromagnetic, strong, weak
Runtime tokens	real, passported engine forces	`attract`, `charge`, `magnetism`, `fieldflow`
Metrics	measured state, never forces	mass, density, attention, entropy, memory
Diagnostics	render / inspection modes	potential, field-lines, topology, causality
Conditions	gates that read state	`active`, `fast`, `slow`, `hot`, `cool`
Patterns	named composites of the above	Priority Well, Evidence Field, Reading Field

The rule is that *a word's lane is never left to the reader to guess*: in the canonical tables the code-styled words are real runtime tokens, and the unstyled words are metrics or diagnostics that can never be authored as a force. `mass` is a metric; `potential` is a diagnostic; neither is a token. This hygiene is exactly what makes patterns auditable (§7.3): because the lanes are distinct and the token lane is strict, concept words can be expressive without ever inventing a force. The lanes are the discipline behind the slogan; the truth modes (§6.3) are the discipline behind the forces.

1.8 7. Authoring: one contract, three surfaces

1.8.1 7.1 Bodies in markup

A body is any element with a `data-body` attribute whose value is one or more space-separated force tokens. The attribute vocabulary is deliberately small:

```
<field-root></field-root>
<h1   data-body="attract" data-strength="1.2" data-feedback>Mass</h1>
<button data-body="repel" data-range="240">Keep clear</button>
```

`data-strength`, `data-range`, `data-spin`, `data-angle`, `data-when` (a condition gate: `active`/`fast`/`slow`/`hot`/`cool`), `data-feedback` (opt into the reverse write-back), `data-color`, `data-absorb`/`data-max` (sink capacity), and `data-preset` (expand a named composite) round out the common cases. Engaging an element — hover, focus, tap — widens its range and amplifies its strength, so *the field answers interaction.*

1.8.2 7.2 Three surfaces, one compiled contract

The same markup is authored three ways, and all three *compile to identical `[data-body]` markup, drive the same `--field-density` feedback variable, and are measured, fed back, and related through the same platform registries:*

- **Native HTML** — the platform runtime attaches to any element with `data-body`; the same markup works unchanged in Astro, Svelte, Vue, or static HTML.
- **Web component** — `<field-root>` (with `<field-field>` as an equivalent alias; prefer `<field-root>` in new code) wraps content and scans for bodies; the platform runtime is the default.
- **React** — `<FieldField>` renders the same contract and maps props to `data-*` tokens; `useFieldField(options)` returns the field handle directly.

That the three surfaces converge on one contract is the practical expression of the architectural claim in §4: the authoring surface is an adapter, and field behavior is independent of it.

1.8.3 7.3 Intent compilation and patterns

Two higher-level authoring layers sit above raw tokens. The **intent compiler** translates a designer-level `data-intent` (with `data-intensity` / `data-risk`) into force tokens and parameters: `draw-focus` compiles to `attract + feedback`, `warn` to `repel + thermal` (with feedback), and so on.

A **pattern** is the reusable unit: *a portable, serializable, inspectable field program.* A `FieldPattern` declares an id, a human intent, an optional natural-field classification, its `primitives` (real tokens), its `bodies` and `relationships`, its `render` layers, its `metrics` and `diagnostics`, a *required* accessibility fallback, and an optional performance budget and expected metrics. Sixty-four patterns ship as validated programs. The decisive property is the *conformance gate*: it rejects any pattern whose primitives are not real passported tokens, whose render layers or diagnostics are not real modes, or whose declared primitives drift from its body tokens. Pattern *prose* may be expressive ("completion releases pressure and decays into memory") while pattern *runtime fields* stay strict (`primitives: [morph, memory, gravity]`), and concept words never invent tokens (spring→`tether`, drag→`viscosity`, reflect→`wall`). Patterns are thus *auditable against the engine's actual capabilities* — the same epistemic discipline as passports, applied to composition.

The three authoring *levels* map cleanly onto these layers: a **designer** works in intents and presets; a **developer** works in `data-body`, render modes, and patterns; an **engine author** writes custom `field()` / `apply()` implementations with conformance.

1.9 8. The reverse half: emergent interface primitives

The forward half of the loop (elements → field) is force application. The *reverse* half (field → elements) is where Fundamental produces behaviors with no direct analogue in CSS or component state. All four primitives below are consequences of the same reciprocal loop, surfaced through the `FeedbackRegistry`.

1.9.1 8.1 Conserved attention

Attention is treated as a *continuous, conserved* field rather than a per-element boolean:

> Selection is a decision. Attention is a field.

In conserved-attention mode, a single finite force budget is shared across the page, `Σ attention_i = A_total`, so engaging one element *pulls* force off the others — emphasis becomes genuinely zero-sum and transfer is *felt* rather than fabricated. This is a direct expression of the conservation invariant (§3.2) at the level of emphasis, and it is the substrate for the alert-fatigue question raised by the System Weather case study (§8.6).

1.9.2 8.2 Material typography

Because the reverse write-back is a continuous scalar (`--field-density`, with `--load` and `--lit` for accretion fill and cross-boundary spill), a *single* density value can drive variable-font weight and optical size, tracking, glow, and color *simultaneously*, in pure CSS:

```
.field-text {
  font-variation-settings: "wght" calc(300 + var(--field-density, 0) * 500);
```

```
  text-shadow: 0 0 calc(var(--field-heat, 0) * 18px) currentColor;
}
```

The governing sentence is: *the field does not decorate the interface; it parameterizes it.* A critical design law accompanies this — the **punctuation rule**: do not assemble particles into words or letterforms (it reads as noisy); reserve particle-into-shape effects for punctuation and marks, and make *words* feel alive by acting on the type and the field around them. *Words are bodies the field decorates; marks are where matter assembles.* Typography, color, shape, and pattern mappings are the subject of the visual-language canon and are summarized in Paper 2.

1.9.3 8.3 Cross-boundary causality

A saturated body spills density to its neighbors, weighted by nearness, surfaced as `--lit`. Emphasis is therefore not strictly contained within an element's own box: a region under heavy attention *lights* its surroundings, producing a spatial, causal coupling between adjacent elements that a purely local state model cannot express. Every such effect remains *traceable to a field cause* — the diagnostic story of §6 and Paper 8 — so the spill can be inspected and explained, not just seen.

1.9.4 8.4 Memory fields

User movement leaves a decaying deposit on a scalar grid, `M(x, y, t+dt) = M(x, y, t)·decay + deposit`, so the interface accumulates *a memory of approach, not just a record of clicks.* Read paragraphs, visited routes, and refined search paths become worn trails that bias future attention. Memory is classified as a *metric* (semantic truth), not a fundamental force — an example of the truth-mode taxonomy keeping the vocabulary honest.

These four are not a closed list; they are the most legible members of a larger possibility space (relationship bonds, evidence fields, presence auras, system "weather") that Papers 2 and 3 develop.

1.9.5 8.5 Reduced-motion equivalence (overview)

A field that expresses meaning through motion would be inaccessible if motion were the *source* of meaning. Fundamental's stance is that **motion is one representation of state, not the meaning itself**, so reduced motion is the *same field differently revealed* rather than a degraded fallback path. Under `prefers-reduced-motion` the runtime swaps travel for state: particles freeze or fade, sparks become static highlights, flow ribbons become static field lines, and heat trails become a soft wash — while the underlying state (attention, memory, relationship) still tracks and is still written to the DOM. Two invariants hold throughout: *meaning is never motion-only*, and *color is never the sole carrier of meaning.* An accessibility preview renders any composition as its reduced surface, which makes these rules verifiable rather than aspirational. We give only the overview here; the formal conformance model — semantic source, visual–semantic binding, motion behavior, static equivalent, and the lint rules that enforce them — is the subject of Paper 4.

1.9.6 8.6 Case studies (overview)

The paradigm is most convincing not in a blank playground but applied to pages users already understand: *here is a page you already know; watch what happens when its meaning becomes a field.* Three case studies anchor the family:

- **Reading Field** (shipped; `/docs/reading-field`). An ordinary long-form article — sections, headings, citations, a table of contents — wired to `createFieldPlatform` and exercising all six registries on the scheduler. Sections are measured bodies; viewport-centre proximity becomes attention; attention accumulates into memory over dwell; the table of contents reflects the field state; citations resolve into typed relationships. Reduced motion preserves the meaning. It proves the substrate on prose, *without spectacle.* Paper 2 turns it into an empirical study.
- **Evidence Field.** Claims, sources, contradictions, confidence, and provenance modeled as inspectable field relationships — support as binding, contradiction as charge, uncertainty as entropy, correction

as memory overwrite. The target domain is AI answers and other source-backed systems where trust primitives are weakest. Paper 3 develops the model and a study.

- **System Weather.** A dashboard whose aggregate field state — urgency, risk, anomaly, coherence — surfaces as "weather" over the interface, with field-based priority as a candidate antidote to alert fatigue.

Together these show the same runtime spanning reading, trust, and operations, each on a familiar interface archetype rather than a bespoke demo.

1.9.7 8.7 Signals-first mode: the field as pure computation layer

The logical extreme of this principle is the **signals-first mode**: `render="none"` (now the engine default, #538) runs the full simulation — forces, registries, feedback, all six scheduler phases — and writes all semantic signals to the DOM (`data-field-density`, `data-field-attention`, `data-field-temperature`, and the corresponding CSS custom properties) *while drawing nothing at all.* The field becomes a pure computation layer: attention measured, memory accreted, relationships tracked, all without a single particle on screen. Visual representation then becomes explicitly opt-in (`render="dots"`, `render="trails"`, etc.) rather than the default. This is not a degraded path — it is a first-class consumer posture. A data-driven agent, a server-side renderer, or any non-visual surface reads the same field state as the visual user sees animated; the physics is the common substrate, representation is a choice above it.

1.10 9. Implementation status and limitations

A preprint earns trust by being precise about what is real. We separate shipped behavior from aspirational behavior and then state the model's deliberate limitations.

1.10.1 9.1 Shipped versus aspirational

Shipped (verifiable in the registry, the manual config, the render-mode catalog, the tests, and the package exports): the five packages; the `FrameScheduler` and all six registries plus `lintPlatform()`; the platform runtime as the default for `<field-root>` (Phase D) with the guarded renderer-agnostic core; all 36 forces and all 16+ render/diagnostic modes (including `topology`, `inspector`, `causality`, `prediction`); signals-first mode (`render="none"` as the engine default, #538 — full simulation with no visual output); the `field.flowTo()/clearFlow()` controlled flow API; the Reading Field demo, accessibility preview, narrative reveal, and PNG/SVG diagnostic export; the native-HTML / web-component / React authoring surfaces; and the Natural Field Translation classification.

Aspirational or opt-in, and labeled as such throughout: first-class inertial mass (opt-in; see below); several "weak-field" transformations (phase, fission) that are conceptual rather than implemented as forces; and the forward-looking frontiers (input agents, GPU compute, multi-root fields, visual authoring tools) that are roadmap items, not shipped behavior.

1.10.2 9.2 Deliberate limitations (the caveat canon)

The following are *design decisions*, not defects, but a paper must state them plainly:

1. **Mass is nominal by default.** The default integrator advances `v += F` (unit mass), so "heavier particles swing wider" is aspirational; first-class inertial mass (`a = F/m`, `m  size`) is opt-in via `FieldOptions.mass`.
2. **Energy is intentionally not conserved.** Friction and heat decay keep the interface calm. The dissipated energy is *accounted for* — rendered as micro-reactions (sparks scaled to the kinetic energy lost, `ΔE`) rather than silently dropped — but the ambient field is a *driven, damped* system by design.
3. **Momentum is only partially conserved** (pairwise in `collide`; the ambient field is damped).
4. **Designed natural, on purpose.** Bounded designed falloff is not inverse-square; the two registers are kept explicit rather than unified (§6.3).

5. **Particle count is the one strong invariant**, conserved except through explicitly budgeted sources/sinks. The system promises count, not energy or momentum.
6. **No user-study evidence yet.** The interface-benefit claims in this series are framed as designs and hypotheses (Paper 3), never as measured outcomes, pending an actual study.

We consider stating these as part of the contribution: the value of an *inspectable* field language is partly that its honesty is mechanical — truth modes, passports, conformance, and lint exist precisely so that the boundary between physics, design, and expression is checkable rather than rhetorical.

1.11 10. Discussion

What the layering buys. The empty-allowlist DOM boundary (§4.2) is more than hygiene: it means the *concept* of Fundamental (a field over participants) is genuinely separable from *the web* as a render target. The same engine could drive a native canvas, a WebGPU compute backend, or a headless analysis. The platform layer is then "just" one — important, carefully disciplined — binding of that engine to one substrate.

What the scheduler buys. Read/write phase separation is the difference between a physics overlay that fights the browser's layout and one that rides it. By reading all geometry once into an immutable snapshot and flushing all writes together, the synchronization invariant holds at animation rates without forced reflow, which is what makes "animating the DOM animates the simulation" tractable rather than pathological.

What the epistemics buy. Truth modes, passports, conformance, and lint are unusual for an interface library. Their payoff is that Fundamental can make *auditable* claims: a reviewer can check that magnetism does no work, that a pattern references only real tokens, that no overlay mutates physics, and that the core reaches no DOM global — by running tests, not by trusting prose. We argue this is the right posture for a system that asks designers to reason about *cause* ("why did this move?") rather than only appearance.

Threats to the framing. The reciprocal model is most compelling on content that *has* latent relational structure (long documents, evidence, dashboards, graphs). On simple, flat interfaces the overhead of bodies and fields may not pay for itself, and the risk of "spectacle over meaning" is real — which is exactly why the roadmap prioritizes coherence, authorability, inspectability, and trust over new visual effects. Whether the model measurably helps users is an open empirical question that Paper 3 is designed to answer.

1.12 11. Conclusion

Fundamental reframes interface state from local and binary to spatial, relational, and reciprocal: *elements bend the field, and the field bends them back.* The system that realizes this reframe is a strictly layered, provably renderer-agnostic engine; a six-phase scheduler and six single-concern registries that keep the DOM field loop consistent without layout thrashing; a force model that separates structure from cause and classifies every behavior by truth mode and passport; and a uniform authoring contract that compiles identically across native HTML, web components, and React. The reciprocal loop yields interface primitives — conserved attention, material typography, cross-boundary causality, and memory fields — that local component state cannot express. We have been explicit about the boundary between what is shipped and what is aspirational, and about the model's deliberate physical limitations. The companion papers validate the paradigm in specific domains: reading (Paper 2), evidence and trust in AI interfaces (Paper 3), reduced-motion equivalence (Paper 4), the host-driven runtime architecture in depth (Paper 5), portable pattern authoring (Paper 6), data binding (Paper 7), and explainable diagnostics (Paper 8).

1.13 Appendix A. Reproducibility and verification

Every architectural claim in this paper is checkable against the repository. The load-bearing anchors:

- Renderer-agnostic core: `packages/core/src/engine/dom-boundary.test.ts` (empty allowlist).
- The scheduler and registries: `packages/dom/src/schedule.ts`, `platform.ts`, `measurement.ts`, `state.ts`, `feedback.ts`, `relationships.ts`, `visual-bindings.ts`, `overlays.ts`, `lint.ts`.
- The force contract and passports: `packages/core/src/engine/registry.ts`, `packages/core/src/contracts/passpo`
- The integrator and the unit-mass / opt-in-mass behavior: `packages/core/src/engine/integrator.ts`.
- The canonical force/feedback catalog: `packages/core/src/config/manual.ts`.
- Authoring/scan and the `[data-body]` contract: `packages/core/src/engine/scanner.ts`.

The canonical design documents corroborate the framing: the system contracts, the platform architecture, the fundamental field-behavior table, and the natural-fields classification under `docs/canonical/`, and the as-built engine record under `docs/engine-reference/`.

1.14 Appendix B. Conversion notes (markdown → preprint)

Notation is kept LaTeX-compatible (inline math and fenced blocks translate directly). Figures referenced in prose but not yet drawn — the package dependency graph (§4.1), the reciprocal loop (§3.1), the six-phase scheduler timeline (§5.1), and a field-vs-particle-path diagram (§3.3) — are produced at conversion time. External citations and bibliography keys are resolved in `references.md` must be resolved and verified before submission.